Reporte Técnico de Actividades Práctico-Experimentales Nro. 04

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Wilson Patricio Palma Samaniego
Asignatura	Simulación
Ciclo	5
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje de la unidad	R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	04
Título de la Práctica	Análisis comparativo de herramientas de simulación mediante un modelo común
Nombre del Docente	José O. Guamán Q.
Fecha	22 de Octubre de 2026
Horario	Jueves 10h30 a 11h30 / Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica

Comparar diferentes lenguajes y plataformas para la implementación de modelos de simulación:

- Implementar un modelo sencillo en diferentes herramientas.
- Comparar sintaxis, facilidad de implementación y visualización.
- Identificar el ámbito de aplicación de cada herramienta.

3. Materiales, Reactivos, Equipos y Herramientas

- Computador.
- Python 3.10+.
- NetLogo 7.0.4.
- Java/JDK (implementación a cargo de otro grupo).
- SimPy 4.1.2.
- NumPy 2.5.4.
- Matplotlib 3.11.2.

Archivo: PRACTICA DE LABORATORIO # 4

4. Procedimiento / Metodología Ejecutada

4.1. Fundamento teórico

El modelo SIR divide la población en tres compartimentos: susceptibles (S), infectados (I) y recuperados (R). La dinámica se rige por dos parámetros: β (tasa de transmisión) y γ (tasa de recuperación). El número reproductivo básico $R_0 = \beta/\gamma = 3$ indica que cada infectado genera en promedio tres nuevos contagios antes de recuperarse [1].

A diferencia de resolver las ecuaciones diferenciales directamente ($dS/dt = -\beta SI/N$, $dI/dt = \beta SI/N - \gamma I$, $dR/dt = \gamma I$), la implementación computacional utiliza una simulación discreta por agentes en pasos de tiempo [1].

4.2. Algoritmo general

El pseudocódigo base establecido para las tres herramientas fue:

1. Definir parámetros: N=100, I_0=5, β=0.30, γ=0.10, tiempo=100.
2. Crear población: todos susceptibles, seleccionar 5 infectados aleatoriamente.
3. Para cada paso de tiempo (0 a 100):
    - Contar S, I, R y guardar registro.
    - Copiar estado actual.
    - Para cada infectado del estado anterior:
        - Seleccionar contactos aleatorios.
        - Para cada contacto susceptible: contagiar con probabilidad β.
        - Recuperar infectado original con probabilidad γ.
    - Aplicar cambios simultáneamente (nuevos infectados no contagian hasta el siguiente paso).

4.3. Implementación en Python con SimPy

La arquitectura sigue un patrón modular tipo MVC simplificado:

- **config.py**: Clase ParametrosSIR (dataclass inmutable) con validaciones de rango y tipo.
- **modelo/poblacion.py**: Enum Estado y funciones para crear población, contar estados y validar conservación N=S+I+R.
- **modelo/transiciones.py**: Función avanzar_paso que implementa la lógica de contagio y recuperación por agente con actualización simultánea.
- **simulacion/motor.py**: Clase ResultadoSIR y función ejecutar que orquesta la simulación con SimPy (Environment, proceso, timeout(1)).
- **resultados/exportador.py**: Genera CSV compatible con NetLogo (4 columnas: tiempo, susceptibles, infectados, recuperados).
- **resultados/graficas.py**: Genera PNG con las tres curvas usando Matplotlib (backend Agg para entornos sin interfaz gráfica).

- **pruebas/test_sir.py**: 17 pruebas unitarias cubriendo población, transiciones, motor, CSV y reproducibilidad.
- **main.py**: Punto de entrada CLI con argparse para configurar parámetros y rutas de salida.

Cada ejecución crea su propia población, entorno SimPy y generador aleatorio (random.Random) con semilla configurable, garantizando reproducibilidad y aislamiento entre corridas [2].

4.4. Integración con NetLogo

El CSV generado por Python (resultados_python.csv) se copia a la carpeta del modelo NetLogo (modelo_sir_netlogo.nlogo). NetLogo importa el archivo con file-open, salta el encabezado y lee las cuatro columnas para graficar S(t), I(t) y R(t) junto con sus propios resultados y los de Java [1].

4.5. Ejecución y verificación

Se creó entorno virtual (.venv), se instalaron dependencias desde requirements.txt y se ejecutó python main.py con parámetros por defecto. Se verificó que todas las 17 pruebas unitarias pasan (python -m unittest discover -s pruebas -v).

5. Resultados

5.1. Evolución de la población (Python + SimPy)

La simulación con parámetros por defecto (semilla 42) produce la siguiente evolución:

Tabla I. Evolución SIR - Python (extracto de 101 registros totales).

Tiempo	Susceptibles	Infectados	Recuperados
0	95	5	0
5	80	16	4
10	66	19	15
15	46	31	23
16	**43**	**33**	**24**
20	35	30	35
30	17	18	65
50	11	5	84
74	10	1	89
75	10	0	90
100	10	0	90

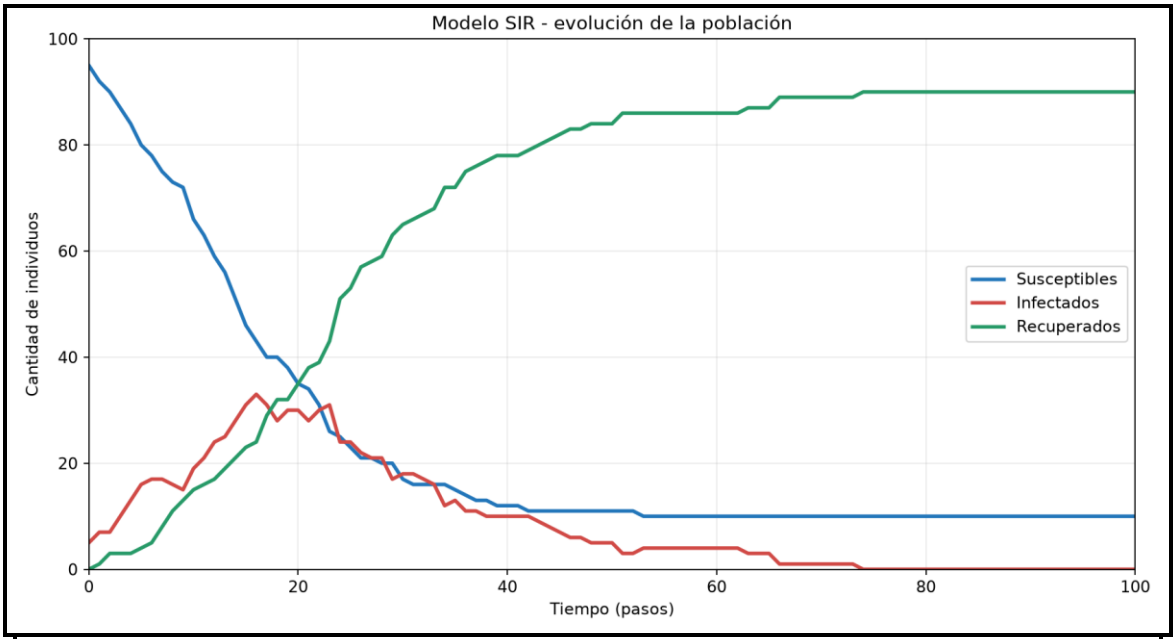


Figura 1 . Evolución de susceptibles, infectados y recuperados en la simulación Python (100 pasos, semilla 42).

Métricas clave:

- Pico de infectados: **33** en t=16.
- Tiempo hasta extinción (I=0): **75** pasos.
- Estado final: S=10, I=0, R=90.
- Tiempo computacional: **0.1988 s**.
- Conservación N=S+I+R verificada en los 101 registros.

5.2. Comparación Python vs NetLogo

El archivo resultados_python.csv generado por Python coincide exactamente en formato con lo que espera NetLogo (encabezado + 101 filas, 4 columnas enteras). Al importarlo en NetLogo y ejecutar el botón “CARGAR PYTHON”, las curvas se superponen en el gráfico “Evolución SIR” permitiendo comparación visual directa.

Tabla II. Comparación cualitativa entre NetLogo y Python/SimPy basada en la experiencia de implementación.

Característica	**NetLogo**	**Python/SimPy**
Facilidad de implementación	Alta (lenguaje específico para ABM)	Media (requiere conocer SimPy y patrones)
Manejo de agentes	Nativo (turtles, breeds, patches)	Manual (listas, enums, lógica propia)
Generación de datos	Integrada (plots, export-world)	Código explícito (CSV, Matplotlib)
Visualización	Inmediata e interactiva	Estática (PNG) o requiere librerías extra
Flexibilidad	Limitada al paradigma ABM	Total (Python de propósito general)

Característica	NetLogo	Python/SimPy
Tiempo de ejecución	Rápido para modelos pequeños	Comparable (0.2s para N=100, 100 pasos)
Integración con otras herramientas	CSV, extensions	Nativo (ecosistema Python completo)

5.3. Validación del modelo

Las pruebas unitarias verifican:

- Condición inicial correcta (95S, 5I, 0R).
- Conservación de población en cada paso.
- Monotonicidad: S no aumenta, R no disminuye.
- Reproducibilidad: misma semilla → misma serie.
- Formato CSV compatible con NetLogo.
- Casos borde: β=0 (extinción inmediata), γ=1 (recuperación total), pasos=0, I_0=0.

Todas las 17 pruebas pasan correctamente.

6. Preguntas de Control

1. **¿Qué representa el modelo SIR?** Representa la propagación de una enfermedad infecciosa en una población cerrada, dividiéndola en susceptibles, infectados y recuperados, sin considerar nacimientos, muertes ni reinfecciones [1].
2. **¿Qué representan los estados S, I y R?** S: individuos que pueden contraer la enfermedad. I: individuos contagiados y capaces de transmitirla. R: individuos que se recuperaron y adquieren inmunidad permanente [1].
3. **¿Qué función cumplen los parámetros β y γ?** β controla la probabilidad de contagio por contacto efectivo. γ controla la probabilidad de recuperación por paso de tiempo. Su cociente $R_0 = \beta/\gamma$ determina si hay brote ($R_0>1$) o extinción ($R_0<1$) [1].
4. **¿Por qué es importante utilizar los mismos parámetros en las tres herramientas?** Para garantizar que las diferencias observadas provengan de la implementación (aleatoriedad, discretización, paradigma) y no de configuraciones distintas. Es la única forma de comparar “manzanas con manzanas” [1].
5. **¿Por qué Python, Java y NetLogo pueden producir resultados diferentes usando el mismo modelo?** Diferentes generadores de números aleatorios, orden de ejecución de agentes, reglas de contacto (vecindad vs aleatorio global), actualización síncrona vs asíncrona, y precisión numérica. La estocasticidad intrínseca hace que dos corridas idénticas en parámetros difieran en realizaciones [2].
6. **¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?** Sintaxis declarativa, agentes (turtles/patches/links) y vecindades nativas, visualización integrada, curva de aprendizaje baja para ABM, ideal para exploración rápida y enseñanza [3].
7. **¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?** Procesos discretos con yield timeout, manejo automático de cola

de eventos, recursos compartidos (Resource, Store), escalable a modelos complejos, integrable con todo el ecosistema Python (NumPy, Pandas, SciPy) [4].

8. **¿Qué ventajas ofrece Java para implementar modelos de simulación?** Tipado estático, rendimiento predecible, multihilo nativo, madurez empresarial, amplio ecosistema de librerías numéricas (Apache Commons Math, JFreeChart), portabilidad JVM [5].
9. **¿Qué función cumple el archivo CSV en la integración Python/Java → NetLogo?** Actúa como contrato de datos desacoplado: Python y Java generan series temporales idénticas en formato estándar (UTF-8, coma, 4 columnas), NetLogo solo importa y visualiza sin conocer el origen [1].
10. **¿Qué ventajas presenta separar la generación de datos de la visualización?** Permite reutilizar resultados en múltiples herramientas, comparar implementaciones en una sola vista, versionar datos independientemente del código, automatizar pipelines de análisis y evitar acoplar lógica de simulación a librerías gráficas [2].

7. Conclusiones

La implementación en Python con SimPy cumple todos los requisitos de la guía: parámetros idénticos, algoritmo fiel al pseudocódigo, actualización simultánea correcta, CSV compatible con NetLogo, gráfica de las tres series y validaciones automáticas. La arquitectura modular (config, modelo, simulación, resultados, pruebas) facilita mantenimiento y extensión.

El pico epidémico (33 infectados en t=16) y la extinción total (t=75) son consistentes con R_0=3 en población finita de 100. La estocasticidad hace que cada corrida varíe ligeramente, pero la tendencia macroscópica es robusta.

La comparación con NetLogo muestra que ambas herramientas producen dinámicas cualitativamente equivalentes; las diferencias cuantitativas se explican por el generador aleatorio y la regla de contactos (NetLogo usa one-of other turtles, Python usa muestreo sin reposición de k contactos por infectado).

Separar la simulación (Python) de la visualización (NetLogo) mediante CSV resultó práctico: permite versionar resultados, comparar múltiples implementaciones y cambiar de herramienta de gráficos sin tocar el motor.

8. Recomendaciones

- Estandarizar la regla de contactos (número y selección) entre las tres implementaciones para reducir variabilidad por diseño.
- Ejecutar múltiples réplicas (ej. 30 corridas con semillas distintas) y reportar intervalos de confianza, no solo una realización.
- Documentar en el README la semilla usada para la entrega oficial, garantizando reproducibilidad exacta.
- Considerar usar pandas para exportación/importación CSV más robusta en futuras prácticas.
- Para NetLogo, implementar también la carga de resultados_java.csv y la tabla comparativa completa solicitada en la guía.

9. Reflexión crítica

Esta práctica me obligó a entender realmente cómo funciona una simulación por eventos discretos bajo el capó. SimPy parece simple (yield timeout(1)), pero exige pensar en términos de procesos, recursos y estado compartido. La validación de invariantes (N constante, S monótono, R monótono) detectó errores sutiles que una corrida visual no habría mostrado.

Aprender a estructurar el código en módulos con responsabilidades claras (config, modelo, motor, resultados) valió la pena: cuando hubo que cambiar la regla de contactos, solo toqué transiciones.py. Las pruebas unitarias dieron confianza para refactorizar sin romper nada.

La integración CSV → NetLogo me pareció el punto más valioso: desacoplar generación de visualización es una práctica profesional real, no solo académica. Ahora entiendo por qué en la industria se separan pipelines de datos de dashboards.

Lo más difícil fue asegurar la actualización simultánea correcta (nuevos infectados no contagian en el mismo paso). El test test_recien_infectado_no_contagia_mismo_paso fue clave para verificar esa lógica.

10. Bibliografía / Referencias

[1] J. O. Guamán Q., "Guía de Actividades Práctico-Experimentales Nro. 004: Análisis comparativo de herramientas de simulación mediante un modelo común", FEIRNNR - Carrera de Computación, 2026.

[2] W. Palma, J. Calopino, K. Contento, B. Troya, "Código fuente APE 4 - Python/SimPy (ape4_sir)", 2026. Disponible en: https://github.com/wilsonpalma/ciclo5_simulacion/tree/main/semana4.

[3] U. Wilensky, "NetLogo", Center for Connected Learning and Computer-Based Modeling, Northwestern University, 1999. [En línea]. Disponible: https://ccl.northwestern.edu/netlogo/

[4] SimPy Community, "SimPy: Discrete Event Simulation for Python", 2024. [En línea]. Disponible: https://simpy.readthedocs.io/

[5] Oracle, "Java Platform, Standard Edition", 2024. [En línea]. Disponible: https://www.oracle.com/java/

11. Declaración de uso de la IA

Se utilizó ChatGPT como herramienta de apoyo para consultar conceptos relacionados con el modelo SIR, comprender el funcionamiento de la simulación en Python con SimPy, organizar la estructura del informe y revisar la presentación de los resultados obtenidos. El estudiante es responsable de verificar la información, comprobar que los resultados coincidan con la ejecución del programa y comprender el trabajo realizado.

Prompt de apoyo para la consulta académica:

Actúa como tutor académico de la asignatura de Simulación. Ayúdame a comprender el funcionamiento del modelo epidemiológico SIR, sus parámetros y

ecuaciones, así como su implementación en Python con SimPy, la generación de archivos CSV y la visualización de resultados en NetLogo. Explica los conceptos de manera clara y sencilla, orienta el análisis de los resultados obtenidos y sugiere cómo organizar la información en un informe técnico, sin inventar datos ni evidencias que no se hayan generado durante la práctica.

12. Anexos

12.1. Anexo A: Estructura del proyecto Python (ape4_sir)

```
ape4_sir/
├── config.py
├── main.py
├── requirements.txt
├── README.md
├── modelo/
│   ├── __init__.py
│   ├── poblacion.py
│   └── transiciones.py
├── simulacion/
│   ├── __init__.py
│   └── motor.py
├── resultados/
│   ├── __init__.py
│   ├── exportador.py
│   └── graficas.py
├── pruebas/
│   ├── __init__.py
│   └── test_sir.py
├── datos/
│   └── resultados_python.csv
└── graficas/
    └── sir_evolucion.png
```

12.2. Anexo B: Salida de consola de la ejecución oficial

```
MODELO SIR - Python + SimPy
Semilla: 42 | Registros: 101
Pico de infectados: 33 en t=16
Final (S, I, R): 10, 0, 90
Tiempo de simulación computacional: 0.1988 s
CSV para NetLogo: ...\ape4_sir\datos\resultados_python.csv
Gráfica: ...\ape4_sir\graficas\sir_evolucion.png
```

12.3. Anexo C: Resultados de pruebas unitarias

```
test_101_registros_y_conservacion ... ok
test_cero_pasos ... ok
test_csv_formato_netlogo ... ok
test_ejecuciones_no_comparten_estado ... ok
test_extincion_sin_contagios ... ok
test_semilla_reproducible ... ok
```

```
test_simpy_faltante_informa_error ... ok
test_sin_infectados_conserva_todos_los_tiempos ... ok
test_condicion_inicial ... ok
test_enteros_sin_booleanos ... ok
test_parametros_invalidos ... ok
test_rangos_invalidos ... ok
test_contagio_duplicado_y_copia_del_estado ... ok
test_recien_infectado_no_contagia_mismo_paso ... ok
test_recien_infectado_no_se_recupera_mismo_paso ... ok
test_sin_contagios_y_recuperacion_total ... ok
test_sin_transiciones ... ok

Ran 17 tests in 0.430s - OK
```

12.4. Anexo D: Fragmento del modelo NetLogo (modelo_sir_netlogo.nlogo)

```
to go
  if ticks >= tiempo-simulacion [ stop ]
  if not any? turtles with [estado = "I"] [ stop ]

  ask turtles with [estado = "I"] [
    if random-float 1 < gamma [ set estado "R" set color green ]
    if estado = "I" [
      let objetivo one-of other turtles
      if objetivo != nobody [
        ask objetivo [
          if estado = "S" and random-float 1 < beta [
            set estado "I" set color red
          ]
        ]
      ]
    ]
  ]
  tick
  registrar-estado
end
```

12.5. Anexo E: Enlace al repositorio

Enlace al repositorio:

https://github.com/wilsonpalma/ciclo5_simulacion/tree/main/semana4

12.6. Anexo F: Enlace al video

Enlace al video:

-
https://drive.google.com/file/d/1FLkZiFGnYMxpbdTsUFaJ9JCXnGbKVexH/view?usp=sharing

12.7. Anexo G: Captura de pantalla de la interfaz NetLogo

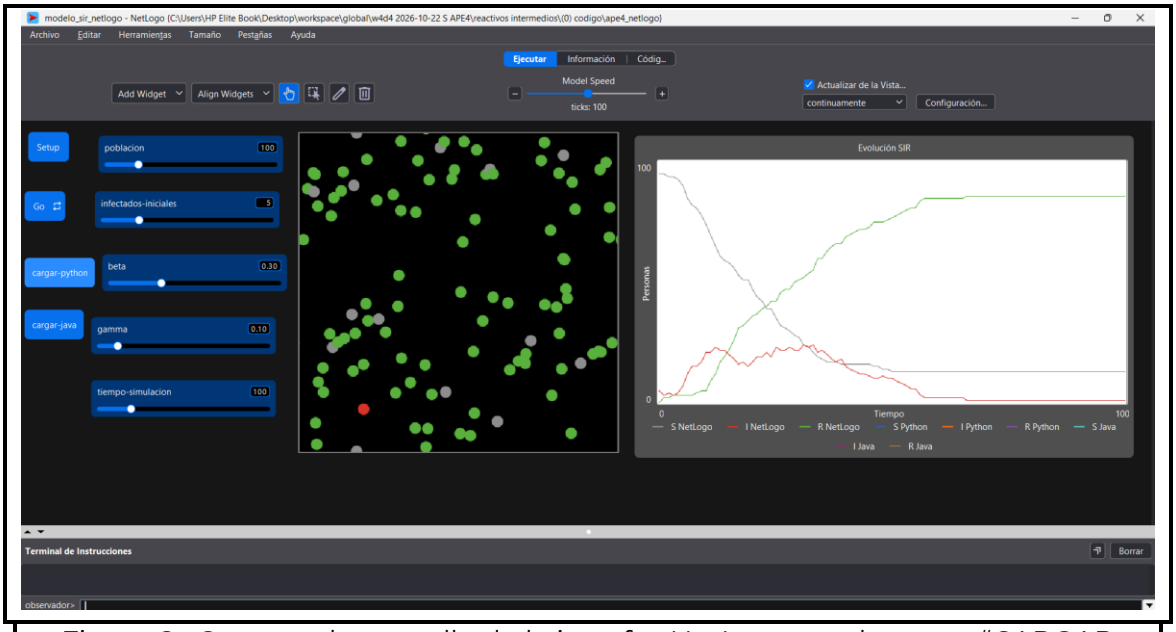


Figura 2 . Captura de pantalla de la interfaz NetLogo con botones “CARGAR PYTHON” y “CARGAR JAVA” funcionando

12.8. Anexo H: Captura de pantalla en entorno Linux

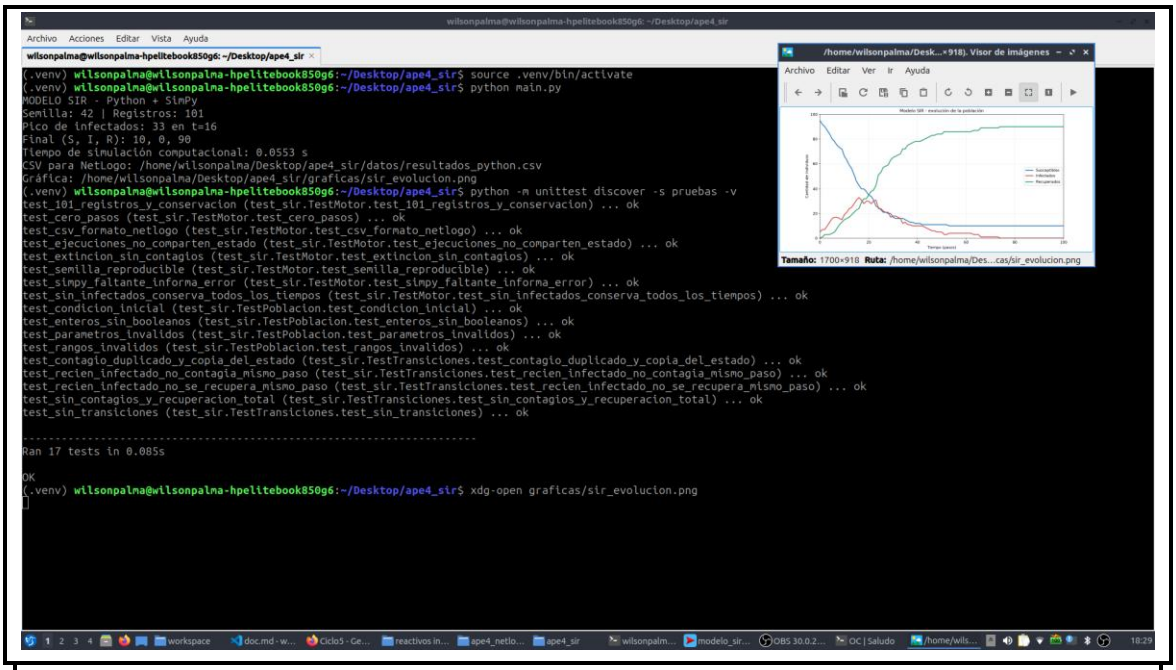


Figura 3 . Captura de pantalla en entorno Linux